REPORT

TASK 1B:

Task 1B

per layer = (nin × nout) + nout

$$\text{Total} = \sum_{\text{layers}} (\text{nin} \times \text{nout} + \text{nout})$$

1) 2 Layers → [2, 8, 1]

Layer 1 → (2 × 8) + 8 = 24

Layer 2 → (8 × 1) + 1 = 9

Total = 24 + 9 = $\boxed{33}$

2) 5 Layers → [2, 8, 8, 8, 8, 8, 1]

Layer 1 → (2 × 8) + 8 = 24

Layer 2 → (8 × 8) + 8 = 72

3 → (8 × 8) + 8 = 72

4 → (8 × 8) + 8 = 72

5 → (8 × 1) + 1 = 9

Total = 24 + 72 + 72 + 72 + 72 + 9 = $\boxed{249}$

3) 10 Layers → [2, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 1]

Layer 1 → (2 × 8) + 8 = 24

2 – 9 → (8 × 8) + 8 = 72 (each)

Layer 10 → (8 × 1) + 1 = 9

Total = 24 + (8 × 72) + 9 = $\boxed{609}$

TASK 1C:

```
Model: 2-layer | Activation: sigmoid | Optimizer: SGD
----------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5684
Final Test Loss           : 0.5764

Model: 2-layer | Activation: sigmoid | Optimizer: Momentum
----------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5684
Final Test Loss           : 0.5762

Model: 2-layer | Activation: relu | Optimizer: SGD
----------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.533
Final Validation Loss     : 0.4876
Final Test Loss           : 0.5097

Model: 2-layer | Activation: relu | Optimizer: Momentum
----------------------------------------------------------------------
Final Training Accuracy   : 0.9548
Final Validation Accuracy : 0.9667
Final Test Accuracy       : 0.9556
Final Training Loss       : 0.1364
Final Validation Loss     : 0.1177
Final Test Loss           : 0.1307
```

```
Model: 5-layer | Activation: sigmoid | Optimizer: SGD
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5685
Final Test Loss           : 0.5762

Model: 5-layer | Activation: sigmoid | Optimizer: Momentum
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5685
Final Test Loss           : 0.5762

Model: 5-layer | Activation: relu | Optimizer: SGD
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6071
Final Validation Loss     : 0.5687
Final Test Loss           : 0.5764

Model: 5-layer | Activation: relu | Optimizer: Momentum
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5685
Final Test Loss           : 0.5762
```

```
Model: 10-layer | Activation: sigmoid | Optimizer: SGD
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5685
Final Test Loss           : 0.5762

Model: 10-layer | Activation: sigmoid | Optimizer: Momentum
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5685
Final Test Loss           : 0.5762

Model: 10-layer | Activation: relu | Optimizer: SGD
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.569
Final Test Loss           : 0.5766

Model: 10-layer | Activation: relu | Optimizer: Momentum
--------------------------------------------------------------------
Final Training Accuracy   : 0.7043
Final Validation Accuracy : 0.7489
Final Test Accuracy       : 0.74
Final Training Loss       : 0.6072
Final Validation Loss     : 0.5685
Final Test Loss           : 0.5762
```

TASK 1C:

The gradient norm plots show that in the 2-layer sigmoid network, gradients remain large across layers. However, in the 10-layer sigmoid network, the gradient at the first layer becomes very small while the last layer gradient is larger. This depicts the vanishing gradient problem in which gradients decrease exponentially as they propagate backward through layers.

Required Analysis:

1. Does increasing depth always improve validation performance?
   No.
2. What happens to test performance as depth increases?
   Test accuracy did not improve and sometimes slightly decreased.
3. Do sigmoid networks degrade faster with depth?
   Yes.
4. Do sigmoid networks degrade faster with depth?
   Yes.
5. Does optimizer choice affect deep networks more than shallow ones?
   Yes.

TASK 2A:

```
Train Accuracy: 1.0
Validation Accuracy: 1.0
Test Accuracy: 1.0
Train Loss: 0.010497529233055613
Validation Loss: 0.010369597730311994
Test Loss: 0.010507607647326994
```

TASK 2C:

```
Dense Parameters: 609
CNN Parameters: 20

Parameter Comparison:
Dense Model: 609
CNN Model: 20
Dense / CNN ratio: 30.45
```

Dense layers connect every input to every neuron, so the total number of parameters grows quickly. CNN shares parameters, the same filter is applied across the whole image. Densenetwork moves with the total number of inputs while CNN moves with filter size. That's why dense model has more parameters than CNN.

TASK 2D:

```
Train Accuracy, Validation Accuracy, Test Accuracy, Train Loss, Validation Loss, Test Loss

CNN with Pooling: (1.0, 1.0, 1.0, 0.0007947614881640881, 0.0006843446511564294, 0.000688841330697944)
CNN without Pooling: (1.0, 1.0, 1.0, 0.00042256982009831003, 0.0003611721216717981, 0.00037211133452908315)
CNN with Dropout: (0.9895238095238095, 1.0, 1.0, 0.019233309512564505, 0.010101317210674341, 0.009853687372970878)
CNN without Dropout: (1.0, 1.0, 1.0, 0.0008180936592674756, 0.0007032124719338521, 0.0007084291821973436)
```

Overfitting Comparison – Models without pooling or dropout showed slightly higher overfitting.

```
CNN + Pool-
Accuracy Gap   : 0.0
Loss Gap       : -0.0001

CNN - Pool-
Accuracy Gap   : 0.0
Loss Gap       : -0.0001

CNN + Dropout-
Accuracy Gap   : -0.0105
Loss Gap       : -0.0091

CNN - No Dropout-
Accuracy Gap   : 0.0
Loss Gap       : -0.0001
```

Required Analysis:

1. Which model generalizes better on test set - dense or CNN?
   CNN does better.
2. Does pooling improve test performance?
   Yes.
3. Does dropout reduce validation-test gap?
   Yes.
4. How does parameter count relate to test accuracy?
   More parameters did not mean better test accuracy. The dense model had more but still CNN outperformed it.
5. Does CNN scale better with larger image size?
   Yes.

TASK 3:

```
SGD
------------------------------------------------------------
Final Train Accuracy : 1.0
Final Val Accuracy   : 1.0
Final Test Accuracy  : 1.0
Final Train Loss     : 0.0009
Final Val Loss       : 0.0007
Final Test Loss      : 0.0007
Convergence Epoch    : 4
Loss Stability (Std) : 0.111283

Momentum
------------------------------------------------------------
Final Train Accuracy : 1.0
Final Val Accuracy   : 1.0
Final Test Accuracy  : 1.0
Final Train Loss     : 0.0001
Final Val Loss       : 0.0
Final Test Loss      : 0.0
Convergence Epoch    : 4
Loss Stability (Std) : 0.012637

Adam
------------------------------------------------------------
Final Train Accuracy : 1.0
Final Val Accuracy   : 1.0
Final Test Accuracy  : 1.0
Final Train Loss     : 0.0
Final Val Loss       : 0.0
Final Test Loss      : 0.0
Convergence Epoch    : 4
Loss Stability (Std) : 0.003189
```

All optimizers achieved perfect accuracy on train, validation, and test, so performance wise they are equivalent. However, Adam showed the most stable training followed by Momentum while SGD had the highest problems. This indicates that although all converge quickly, Adam provides the most stable optimization.

FINAL REFLECTION:

1. Where did training fail due to structure?
   Deep sigmoid networks (10 layers) because the gradients vanished, so no learning.
2. Where did optimizer matter more than activation?
   In dee CNN, optimizer mattered more since all used relu but differed in stability.
3. Where did activation matter more than depth?
   In deep networks, activation mattered more because deep sigmoid networks were degrading.
4. What causes gradient shrinkage?
   Repeated multiplication of small derivatives shrinks it across many layers.
5. Why does CNN generalize better than dense?
   CNN uses parameter sharing, so it uses fewer parameters as compared to dense layers.
6. Why does dropout reduce overfitting?
   Because it randomly disables neurons during training which forces it to learn robust featured instead of just learning it.
7. When does depth hurt test performance?
   When the gradient vanishes and the model is no more learning anything.
8. Did validation always predict test performance correctly?
   Yes, mostly.